

Unidad VII: Grafos

Clase 13: Resolver problemas con Grafos

Objetivo

Aplicar grafos a problemas integrales con estrategia y modularidad.

Tema

- Operaciones del TDA Grafo.
- Resolución de problemas completos.
- Análisis de costos.

Resolución de problemas con Grafos

Introducción conceptual

Los grafos permiten representar problemas donde los elementos no se organizan en una jerarquía rígida, sino en una red de relaciones. Esta diferencia es fundamental respecto de los árboles. En un árbol existe una raíz, una estructura padre-hijo y caminos descendentes. En un grafo, en cambio, un elemento puede conectarse con muchos otros, pueden existir rutas alternativas, vínculos circulares, relaciones unidireccionales y caminos indirectos.

Esta estructura resulta especialmente útil para modelar redes de transporte, sistemas bancarios, plataformas digitales, procesos de auditoría, dependencias entre módulos de software, redes sociales, circuitos logísticos y sistemas de recomendación. En todos estos casos, el interés no está solamente en saber qué elementos existen, sino cómo se conectan, si un destino es alcanzable, qué camino permite llegar y cuál es la ruta más conveniente según un criterio de costo, distancia, tiempo, riesgo o prioridad.

El estudio de grafos combina dos dimensiones. La primera es estructural: cómo se representa la red mediante vértices y aristas, y cómo se implementa el Tipo de Dato Abstracto Grafo. La segunda es algorítmica: cómo se recorren esos vértices, cómo se evitan ciclos, cómo se determina la alcanzabilidad y cómo se calculan caminos de menor costo. Esta segunda dimensión es central para resolver problemas completos, porque muchas preguntas reales no se responden con una única consulta directa, sino con un recorrido o con una estrategia de optimización.

El grafo como estructura de relaciones

Un grafo es una estructura compuesta por vértices y aristas. Los vértices, también llamados nodos, representan las entidades del problema. Las aristas representan las relaciones entre esas entidades. Formalmente, un grafo suele expresarse como:

$G = (V, A)$

donde V es el conjunto de vértices y A es el conjunto de aristas.

En un sistema de logística, los vértices pueden ser ciudades, depósitos o centros de distribución, mientras que las aristas pueden representar rutas disponibles. En una plataforma bancaria, los vértices pueden ser cuentas y las aristas pueden representar transferencias. En un sistema de software, los vértices pueden ser módulos y las aristas pueden indicar dependencias.

Cuando una arista tiene sentido, el grafo es dirigido. Una arista $A \rightarrow B$ indica que existe una relación desde A hacia B , pero no necesariamente desde B hacia A . Esto es importante porque muchos problemas reales no son recíprocos. Un vuelo desde Buenos Aires hacia Mendoza no garantiza un vuelo directo de Mendoza hacia Buenos Aires. Una cuenta puede transferir dinero a otra sin que exista una transferencia inversa. Un módulo de software puede depender de otro, pero esa dependencia no funciona necesariamente en sentido contrario.

Cuando una arista tiene un valor asociado, el grafo es ponderado. Ese valor se denomina peso y puede representar distancia, costo, tiempo, riesgo, prioridad, latencia o cualquier medida relevante para comparar conexiones. En una red de transporte, el peso puede ser el costo operativo o los kilómetros recorridos. En una red bancaria, puede representar el monto transferido o el nivel de riesgo. En una red de sistemas, puede representar tiempo de respuesta o carga de comunicación.

La presencia de pesos permite formular preguntas más complejas. No solo interesa saber si existe una ruta directa entre dos puntos, sino cuánto cuesta llegar, si conviene tomar un camino indirecto o cuál es la alternativa más eficiente.

Por ejemplo:

$A \rightarrow B$ costo 100

$A \rightarrow C$ costo 10

$C \rightarrow B$ costo 20

Existe una conexión directa desde A hacia B , pero su costo es 100. También existe un camino indirecto $A \rightarrow C \rightarrow B$, cuyo costo total es 30. En este caso, el camino más conveniente no es el más directo. Esta diferencia justifica el estudio de algoritmos de búsqueda y caminos mínimos.

Operaciones primitivas y algoritmos derivados

El Tipo de Dato Abstracto Grafo, o GrafoTDA, define un conjunto de operaciones básicas que permiten construir, modificar y consultar la estructura. Estas operaciones forman el contrato del grafo. Indican qué se puede hacer con la estructura, sin obligar a conocer cómo está implementada internamente.

Una especificación habitual del GrafoTDA incluye operaciones como:

```
public interface GrafoTDA {
    void InicializarGrafo();
    void AgregarVertice(int v);
    void EliminarVertice(int v);

    ConjuntoTDA Vertices();

    void AgregarArista(int v1, int v2, int peso);
    void EliminarArista(int v1, int v2);

    boolean ExisteArista(int v1, int v2);
    int PesoArista(int v1, int v2);
}
```

Estas operaciones permiten resolver consultas puntuales. Por ejemplo, `ExisteArista(10, 20)` permite saber si existe una conexión directa desde el vértice 10 hacia el vértice 20. `PesoArista(10, 20)` permite obtener el peso de esa conexión directa, siempre que la arista exista.

Sin embargo, muchas preguntas sobre grafos no se responden con una única operación primitiva. Preguntar si existe una arista directa entre A y B no es lo mismo que preguntar si B es alcanzable desde A. Puede no existir una arista directa y, aun así, existir un camino indirecto. Por ejemplo:

```
A -> C
C -> D
D -> B
```

En este caso, `ExisteArista(A, B)` devuelve falso, pero sí existe un camino desde A hasta B. Para responder esta segunda pregunta se necesita recorrer el grafo.

Por eso conviene distinguir dos niveles:

- Operaciones primitivas: consultas o modificaciones directas sobre el TDA.
- Algoritmos derivados: procedimientos que usan las operaciones primitivas para resolver problemas más amplios.

Calcular el grado positivo de un vértice, obtener su vecindario, determinar si existe un camino, recorrer los vértices alcanzables o calcular una ruta de menor costo son algoritmos derivados. No reemplazan al GrafoTDA; se apoyan en él.

Representación interna y criterio de implementación

El mismo GrafoTDA puede implementarse de distintas formas. Dos representaciones habituales son la matriz de adyacencia y la lista de adyacencia.

La matriz de adyacencia utiliza una matriz cuadrada. Las filas representan vértices de origen y las columnas representan vértices de destino. Si existe una arista desde el

vértice de la fila hacia el vértice de la columna, la celda contiene el peso de la arista. Si no existe conexión, la celda puede contener cero o un valor especial que indique ausencia de arista.

Por ejemplo, si la celda `matriz[origen][destino]` contiene 700, eso significa que existe una arista desde el origen hacia el destino con peso 700. La ventaja principal de esta representación es la consulta directa. Saber si existe una conexión entre dos vértices puede resolverse accediendo a una posición de la matriz. Su desventaja es el uso de memoria, porque requiere espacio para todas las combinaciones posibles de vértices, aunque muchas no tengan aristas reales.

La lista de adyacencia asocia a cada vértice una lista de sus vecinos directos. En un grafo ponderado, cada elemento de la lista puede guardar el destino y el peso de la arista. Esta representación es más eficiente en memoria cuando el grafo es disperso, es decir, cuando existen muchos vértices pero pocas conexiones reales. Además, resulta natural para recorrer vecinos. Su desventaja aparece cuando se necesita verificar rápidamente si existe una arista específica entre dos vértices, porque puede ser necesario recorrer la lista del vértice origen.

La elección de representación debe responder al uso esperado:

- Matriz de adyacencia: conviene cuando el grafo es pequeño o denso, o cuando se consultan frecuentemente conexiones directas entre pares de vértices.
- Lista de adyacencia: conviene cuando el grafo es disperso, cuando interesa ahorrar memoria o cuando el problema se basa principalmente en recorrer vecinos.

Desde el punto de vista del diseño, esta diferencia refuerza la importancia del TDA. Un método correctamente escrito debería poder usar GrafoTDA sin depender de si la implementación interna usa matriz o lista. Esa separación permite razonar en términos de comportamiento abstracto, no de detalles internos.

Adyacencia, grado y vecindario

Dos vértices son adyacentes cuando existe una arista entre ellos. En grafos dirigidos, la dirección debe considerarse con precisión. Si existe una arista $A \rightarrow B$, entonces B es vecino saliente de A. Eso no implica que A sea vecino saliente de B.

El grado de un vértice mide la cantidad de conexiones asociadas a ese vértice. En grafos dirigidos se distinguen dos conceptos:

- Grado positivo: cantidad de aristas salientes desde un vértice.
- Grado negativo: cantidad de aristas entrantes hacia un vértice.

En una red de vuelos, si Buenos Aires tiene vuelos hacia Córdoba, Mendoza e Iguazú, su grado positivo es 3. Si recibe vuelos desde Bariloche y Salta, su grado negativo es 2.

El vecindario de un vértice está formado por los vértices alcanzables directamente desde él mediante una arista saliente. Si se trabaja con un conjunto de vértices base, el vecindario del conjunto está formado por todos los vértices que son destino directo de al menos uno de los vértices del conjunto base.

Un método derivado para obtener los vecinos salientes de un vértice puede escribirse recorriendo todos los vértices del grafo y consultando si existe una arista desde el vértice origen hacia cada posible destino:

```
public ConjuntoTDA vecinosSalientes(GrafoTDA g, int origen)
{
    ConjuntoTDA vecinos = new ConjuntoLD();
    vecinos.InicializarConjunto();

    ConjuntoTDA vertices = g.Vertices();

    while (!vertices.ConjuntoVacio()) {
        int destino = vertices.Elegir();
        vertices.Sacar(destino);

        if (g.ExisteArista(origen, destino)) {
            vecinos.Agregar(destino);
        }
    }

    return vecinos;
}
```

Este método no accede a la matriz ni a la lista interna del grafo. Utiliza únicamente operaciones del GrafoTDA. Por eso se mantiene independiente de la implementación.

Con una lógica similar puede calcularse el grado positivo:

```
public int gradoPositivo(GrafoTDA g, int v) {
    int contador = 0;
    ConjuntoTDA vertices = g.Vertices();

    while (!vertices.ConjuntoVacio()) {
        int w = vertices.Elegir();
        vertices.Sacar(w);

        if (g.ExisteArista(v, w)) {
            contador++;
        }
    }

    return contador;
}
```

El grado negativo invierte el sentido de la consulta:

```
public int gradoNegativo(GrafoTDA g, int v) {
    int contador = 0;
    ConjuntoTDA vertices = g.Vertices();

    while (!vertices.ConjuntoVacio()) {
        int w = vertices.Elegir();
        vertices.Sacar(w);

        if (g.ExisteArista(w, v)) {
            contador++;
        }
    }

    return contador;
}
```

Un vértice aislado es aquel que no tiene aristas entrantes ni salientes. Puede detectarse verificando que ambos grados sean cero:

```
public boolean esAislado(GrafoTDA g, int v) {
    return gradoPositivo(g, v) == 0 && gradoNegativo(g, v) == 0;
}
```

Este tipo de métodos muestra una idea importante: resolver problemas con grafos suele consistir en construir operaciones más expresivas a partir de operaciones simples.

Camino, alcanzabilidad y ciclos

Un camino es una sucesión ordenada de vértices donde cada par consecutivo está conectado por una arista. Si existe el camino:

A -> C -> D -> B

entonces B es alcanzable desde A, aunque no exista una arista directa A -> B.

La alcanzabilidad permite responder preguntas como: “¿puede una ciudad llegar a otra mediante conexiones intermedias?”, “¿un usuario está conectado indirectamente con otro?”, “¿una falla en un sistema puede impactar en otro módulo?”, “¿una cuenta investigada puede transferir fondos hacia otra mediante intermediarios?”.

El problema central al recorrer grafos es la existencia de ciclos. Un ciclo es un camino que comienza y termina en el mismo vértice. Por ejemplo:

A -> B -> C -> A

Si un algoritmo recorre un grafo sin registrar qué vértices ya fueron visitados, puede entrar en una repetición indefinida. Desde A puede ir a B, desde B a C, desde C volver a A, y repetir el mismo recorrido una y otra vez.

Por esta razón, los recorridos de grafos necesitan una estructura auxiliar de visitados. Esa estructura suele implementarse con un ConjuntoTDA, donde se agregan los vértices ya descubiertos o procesados.

La regla práctica es clara: antes de avanzar hacia un vértice vecino, se debe verificar si ya fue visitado. Si ya fue visitado, no se lo vuelve a recorrer. Esto evita ciclos infinitos y reduce trabajo repetido.

BFS: búsqueda en anchura

BFS significa Breadth-First Search, traducido como búsqueda o recorrido en anchura. Su estrategia consiste en recorrer el grafo por niveles. Primero se visita el vértice de origen, luego todos sus vecinos directos, después los vecinos de esos vecinos, y así sucesivamente.

La estructura auxiliar natural para BFS es una cola. Una cola respeta el criterio FIFO, First In, First Out, es decir, primero en entrar, primero en salir. Esto permite procesar los vértices en el mismo orden en que fueron descubiertos.

La lógica conceptual de BFS es:

Marcar el origen como visitado.

Acolar el origen.

Mientras la cola no esté vacía:

 desacolar un vértice actual.

 revisar todos sus vecinos.

 por cada vecino no visitado:

 marcarlo como visitado.

 acolarlo.

Una implementación posible utilizando GrafoTDA, ConjuntoTDA y ColaTDA es:

```
public ConjuntoTDA bfs(GrafoTDA g, int origen) {
    ConjuntoTDA visitados = new ConjuntoLD();
    visitados.InicializarConjunto();

    ColaTDA pendientes = new ColaLD();
    pendientes.InicializarCola();

    visitados.Agregar(origen);
    pendientes.Acolar(origen);

    while (!pendientes.ColaVacía()) {
        int actual = pendientes.Primer();
        pendientes.Desacolar();

        ConjuntoTDA vertices = g.Vertices();
```

```

while (!vertices.ConjuntoVacio()) {
    int vecino = vertices.Elegir();
    vertices.Sacar(vecino);

    if (g.ExisteArista(actual, vecino) &&
        !visitados.Pertenece(vecino)) {
        visitados.Agregar(vecino);
        pendientes.Acolar(vecino);
    }
}

return visitados;
}

```

Este método devuelve el conjunto de vértices alcanzables desde el origen. La marca de visitado se realiza cuando el vértice se descubre, no cuando se procesa. Esta decisión evita que el mismo vértice sea acolado varias veces si varios caminos llegan a él.

BFS es especialmente útil cuando interesa conocer la menor cantidad de pasos o saltos en un grafo no ponderado. En una red social, permite encontrar contactos a uno, dos o tres grados de separación. En una red de vuelos, permite determinar qué ciudades son alcanzables con una escala, dos escalas o más, siempre que todas las conexiones se consideren equivalentes.

También puede utilizarse para determinar si existe camino entre dos vértices:

```

public boolean existeCaminoBFS(GrafoTDA g, int origen, int destino) {
    ConjuntoTDA visitados = bfs(g, origen);
    return visitados.Pertenece(destino);
}

```

Esta versión es simple porque reutiliza el recorrido completo. Una versión más eficiente podría cortar apenas encuentra el destino, evitando explorar el resto del grafo. Ese corte anticipado es útil cuando solo interesa una respuesta booleana.

DFS: búsqueda en profundidad

DFS significa Depth-First Search, traducido como búsqueda o recorrido en profundidad. Su estrategia consiste en avanzar por un camino tanto como sea posible antes de retroceder y explorar otras alternativas. Este retroceso se conoce como backtracking.

DFS puede implementarse con una pila o mediante recursividad. En ambos casos, la lógica responde al comportamiento LIFO, Last In, First Out, es decir, último en entrar, primero en salir. La recursividad logra este comportamiento mediante la pila de llamadas del lenguaje.

La lógica conceptual de DFS es:

- Visitar el vértice actual.
- Marcarlo como visitado.

Recorrer sus vecinos.

Por cada vecino no visitado:

aplicar DFS sobre ese vecino.

Una implementación recursiva posible es:

```
public void dfs(GrafoTDA g, int actual, ConjuntoTDA visitados) {
    visitados.Agregar(actual);

    ConjuntoTDA vertices = g.Vertices();

    while (!vertices.ConjuntoVacio()) {
        int vecino = vertices.Elegir();
        vertices.Sacar(vecino);

        if (g.ExisteArista(actual, vecino) &&
            !visitados.Pertenece(vecino)) {
            dfs(g, vecino, visitados);
        }
    }
}
```

Para determinar si existe un camino entre dos vértices, puede construirse un método auxiliar que inicialice el conjunto de visitados y luego invoque la búsqueda:

```
public boolean existeCaminoDFS(GrafoTDA g, int origen, int destino) {
    ConjuntoTDA visitados = new ConjuntoLD();
    visitados.InicializarConjunto();

    dfs(g, origen, visitados);

    return visitados.Pertenece(destino);
}
```

DFS es adecuado cuando interesa explorar caminos completos, analizar dependencias profundas o recorrer estructuras donde importa avanzar por una cadena antes de revisar alternativas. En un sistema de software, puede servir para estudiar dependencias entre módulos. En auditoría de procesos, puede usarse para seguir una cadena completa de eventos, controles o aprobaciones.

Sin embargo, DFS no garantiza encontrar el camino más corto ni el camino de menor costo. Encuentra caminos posibles y permite explorar el grafo, pero el primer camino encontrado puede no ser el más conveniente.

Diferencias prácticas entre BFS y DFS

BFS y DFS son algoritmos de recorrido. Ambos permiten explorar grafos, ambos requieren controlar visitados y ambos pueden determinar alcanzabilidad. La diferencia está en la estrategia.

BFS explora primero lo cercano. Recorre por niveles y utiliza una cola. Por eso resulta natural cuando interesa la menor cantidad de saltos en un grafo no ponderado. Si una

red de vuelos se analiza sin costos y solo se quiere minimizar la cantidad de escalas, BFS es una opción adecuada.

DFS explora primero lo profundo. Avanza por un camino hasta donde puede y luego retrocede. Utiliza recursividad o pila. Es útil para analizar caminos completos, dependencias encadenadas o estructuras donde interesa profundizar antes de abrir alternativas.

La diferencia puede verse con un ejemplo. Supóngase el siguiente grafo:

A -> B
A -> C
B -> D
C -> E
D -> F

BFS desde A tenderá a visitar primero B y C, luego D y E, y finalmente F. DFS desde A podría avanzar por B, luego D, luego F, y recién después volver para explorar C y E.

La elección entre BFS y DFS depende de la pregunta. Para saber si algo es alcanzable, ambos pueden servir. Para menor cantidad de pasos, BFS es más apropiado. Para explorar cadenas completas de dependencia, DFS suele ser más natural.

Cuando los pesos importan: mejores rutas y caminos de menor costo

Los recorridos BFS y DFS tratan las aristas principalmente como conexiones. Pueden responder si un vértice es alcanzable o en qué orden se explora el grafo. Pero cuando las aristas tienen pesos distintos, aparece un problema adicional: encontrar la ruta de menor costo acumulado.

El costo de un camino es la suma de los pesos de sus aristas. Si existe el camino:

A -> C costo 10
C -> B costo 20
entonces el costo total de A -> C -> B es 30.

La ruta más conveniente depende del significado del peso. Si el peso representa costo económico, se busca la ruta más barata. Si representa tiempo, se busca la ruta más rápida. Si representa riesgo, se busca la ruta menos riesgosa. Por eso conviene hablar de “menor costo acumulado” y no de “mejor ruta” en forma absoluta. La mejor ruta siempre depende del criterio que se haya modelado como peso.

BFS no resuelve correctamente el menor costo cuando los pesos son distintos. BFS puede encontrar el camino con menos aristas, pero ese camino puede ser más caro. Por ejemplo:

A -> B costo 100

A -> C costo 10

C -> B costo 20

BFS puede detectar rápidamente que B está a un salto desde A. Sin embargo, el camino directo cuesta 100. El camino A -> C -> B tiene dos saltos, pero cuesta 30. Si se busca menor costo, la segunda ruta es mejor.

Para resolver este tipo de problema se utiliza el algoritmo de Dijkstra, siempre que los pesos sean positivos.

Dijkstra: camino mínimo con pesos positivos

Dijkstra es un algoritmo para calcular caminos de menor costo desde un vértice origen hacia los demás vértices de un grafo ponderado con pesos no negativos. Su idea central es mantener, para cada vértice, la mejor distancia conocida desde el origen. Esa distancia puede mejorar a medida que se descubren nuevos caminos.

El algoritmo utiliza tres estructuras conceptuales:

- **distancia:** guarda el menor costo conocido desde el origen hasta cada vértice.
- **anterior:** guarda el vértice previo en el mejor camino conocido, para poder reconstruir la ruta.
- **visitados:** indica qué vértices ya fueron cerrados, es decir, cuáles ya tienen definida su menor distancia.

La lógica general es:

Inicializar la distancia del origen en 0.

Inicializar la distancia de los demás vértices en infinito.

Mientras queden vértices sin visitar:

 elegir el vértice no visitado con menor distancia conocida.

 marcarlo como visitado.

 revisar sus vecinos.

 si pasar por el vértice actual mejora la distancia hacia un vecino:

 actualizar la distancia del vecino.

 registrar el vértice actual como anterior del vecino.

La operación de mejora se conoce como relajación. Relajar una arista significa revisar si el camino hacia un vecino mejora pasando por el vértice actual.

Por ejemplo:

distancia[A] = 0

distancia[C] = 10

peso(C, B) = 20

Si se está procesando C, se calcula:

```
nuevoCosto = distancia[C] + peso(C, B)
nuevoCosto = 10 + 20
nuevoCosto = 30
```

Si la distancia conocida hacia B era 100, se actualiza:

```
distancia[B] = 30
anterior[B] = C
```

Esto significa que, hasta el momento, la mejor forma de llegar a B es pasar por C.

Estructura implementable de Dijkstra

En una implementación didáctica, puede trabajarse con arreglos auxiliares. Como los vértices del grafo pueden no ser consecutivos, conviene copiar los vértices a un arreglo y usar funciones auxiliares para mapear cada etiqueta de vértice a una posición interna.

La estructura auxiliar puede ser:

```
int[] vertices;
```

```
int[] distancia;
```

```
int[] anterior;
```

```
boolean[] visitado;
```

El arreglo vertices guarda las etiquetas reales de los vértices. El arreglo distancia guarda el menor costo conocido. El arreglo anterior guarda el vértice previo en el camino. El arreglo visitado indica si el vértice ya fue cerrado.

Una versión conceptual del algoritmo puede expresarse así:

```
public void dijkstra(GrafoTDA g, int origen) {
    int INF = 999999;

    ConjuntoTDA conjVertices = g.Vertices();
    int n = contar(conjVertices);

    int[] vertices = pasarAArreglo(g.Vertices(), n);
    int[] distancia = new int[n];
    int[] anterior = new int[n];
    boolean[] visitado = new boolean[n];

    for (int i = 0; i < n; i++) {
        distancia[i] = INF;
        anterior[i] = -1;
        visitado[i] = false;
    }
}
```

```

    }

    int posOrigen = indiceDe(vertices, n, origen);
    distancia[posOrigen] = 0;

    for (int i = 0; i < n; i++) {
        int posActual = menorNoVisitado(distancia, visitado, n);

        if (posActual == -1) {
            return;
        }

        visitado[posActual] = true;
        int actual = vertices[posActual];

        for (int j = 0; j < n; j++) {
            int vecino = vertices[j];

            if (!visitado[j] && g.ExisteArista(actual, vecino)) {
                int nuevoCosto = distancia[posActual] +
g.PesoArista(actual, vecino);

                if (nuevoCosto < distancia[j]) {
                    distancia[j] = nuevoCosto;
                    anterior[j] = actual;
                }
            }
        }
    }
}

```

Este código muestra la estructura esencial del algoritmo. Para una implementación completa, los métodos auxiliares contar, pasarAArreglo, indiceDe y menorNoVisitado deben estar definidos. La modularidad es importante porque cada método debe cumplir una tarea simple.

El método menorNoVisitado busca el vértice no visitado con menor distancia conocida:

```

public int menorNoVisitado(int[] distancia, boolean[] visitado, int n)
{
    int posMenor = -1;
    int menor = 999999;

    for (int i = 0; i < n; i++) {
        if (!visitado[i] && distancia[i] < menor) {
            menor = distancia[i];
            posMenor = i;
        }
    }

    return posMenor;
}

```

Esta versión de Dijkstra no usa una cola de prioridad. Es más simple de comprender y se apoya en arreglos auxiliares. Su costo principal es cuadrático, porque en cada iteración se busca el menor vértice no visitado recorriendo el arreglo de distancias, y luego se revisan posibles vecinos.

Reconstrucción de la ruta

Calcular el costo mínimo no siempre alcanza. En muchos problemas también interesa saber cuál es el camino. Para eso se utiliza el arreglo anterior.

Si el mejor camino encontrado hacia B es:

A -> C -> B

entonces puede quedar registrado así:

anterior[B] = C

anterior[C] = A

anterior[A] = -1

Para reconstruir la ruta, se parte desde el destino y se retrocede por los anteriores hasta llegar al origen. Como ese recorrido se obtiene en orden inverso, puede usarse una pila para invertirlo.

Conceptualmente:

actual = destino

mientras actual no sea -1:

 apilar actual

 actual = anterior[actual]

Al desapilar, los vértices aparecen desde el origen hasta el destino.

Esta idea es útil en problemas de logística, navegación y análisis de procesos. No basta con decir que el costo mínimo desde el depósito 10 hasta la sucursal 80 es 350; también puede ser necesario informar que la ruta es:

10 -> 25 -> 60 -> 80

Comparación entre conexión directa, alcanzabilidad y costo mínimo

Una confusión frecuente consiste en tratar como equivalentes tres preguntas distintas:

- ¿Existe una arista directa?
- ¿Existe algún camino?
- ¿Cuál es el camino de menor costo?

Estas preguntas no son iguales.

ExisteArista(A, B) responde si hay una conexión directa desde A hacia B. Es una consulta puntual.

BFS o DFS responden si B es alcanzable desde A, aunque el camino tenga vértices intermedios. Son algoritmos de recorrido.

Dijkstra responde cuál es el menor costo acumulado desde A hacia B, considerando los pesos de las aristas. Es un algoritmo de optimización de caminos.

La diferencia puede verse con este ejemplo:

A -> B costo 100

A -> C costo 10

C -> B costo 20

Existe arista directa de A a B: sí.

Existe camino de A a B: sí.

El camino de menor costo es A -> C -> B, no A -> B.

Por lo tanto, elegir el algoritmo depende de la pregunta. Si solo interesa conexión directa, alcanza con ExisteArista. Si interesa alcanzabilidad, se usa BFS o DFS. Si interesa menor costo ponderado, se usa Dijkstra.

Análisis de costos

El análisis de costos permite estimar cómo crece el tiempo de ejecución cuando aumenta la cantidad de vértices o aristas. En grafos suelen usarse dos variables:

n: cantidad de vértices.

m: cantidad de aristas.

Cuando se trabaja únicamente con las operaciones del GrafoTDA y no se dispone de una operación directa para obtener los vecinos de un vértice, es habitual recorrer todos los vértices y consultar ExisteArista(actual, vecino). En ese caso, muchos algoritmos tienen una estructura de doble recorrido.

Por ejemplo, para calcular los vecinos de un vértice se recorren todos los vértices posibles. El costo base es $O(n)$. Para calcular el vecindario de un conjunto de hasta n vértices, puede recorrerse el conjunto base y, por cada vértice base, recorrer todos los vértices del grafo. El costo base puede llegar a $O(n^2)$.

BFS y DFS, implementados con GrafoTDA mediante la estrategia de revisar todos los vértices para descubrir vecinos, también pueden analizarse como $O(n^2)$ en el peor caso. Esto se debe a que cada vez que se procesa un vértice se revisan todos los posibles destinos.

En una implementación interna con lista de adyacencia y acceso directo a vecinos, BFS y DFS pueden expresarse como $O(n + m)$, porque procesan vértices y aristas existentes.

Pero si el TDA no expone directamente las listas de vecinos, y el algoritmo debe probar contra todos los vértices, el análisis práctico se acerca a $O(n^2)$.

Dijkstra, en su versión didáctica con arreglos y sin cola de prioridad, tiene costo $O(n^2)$. En cada paso busca el vértice no visitado con menor distancia y revisa sus posibles vecinos. Existen implementaciones más eficientes con cola de prioridad, pero la versión cuadrática es adecuada para comprender la lógica del algoritmo y resolver problemas de tamaño moderado.

Criterios para seleccionar la estrategia de solución

La selección de la estrategia depende de la pregunta del problema. No existe un único algoritmo “mejor” para grafos; existe un algoritmo adecuado para cada tipo de consulta.

Si se necesita saber si dos vértices están conectados directamente, conviene usar ExisteArista. Si además se necesita el valor de esa conexión, se usa PesoArista.

Si se necesita obtener vecinos directos, grados o nodos aislados, se construyen métodos derivados que recorren el conjunto de vértices y consultan la existencia de aristas.

Si se necesita saber si un destino es alcanzable desde un origen, se utiliza BFS o DFS con conjunto de visitados.

Si se necesita encontrar el camino con menor cantidad de saltos en un grafo no ponderado, BFS es la opción natural.

Si se necesita encontrar el camino de menor costo acumulado en un grafo ponderado con pesos positivos, corresponde usar Dijkstra.

Si el problema tiene muchos vértices y pocas aristas, una lista de adyacencia suele ser más eficiente en memoria. Si el problema requiere muchas consultas directas entre pares de vértices y el grafo es denso o pequeño, una matriz de adyacencia puede resultar conveniente.

Errores frecuentes al resolver problemas con grafos

Un error habitual es olvidar que las aristas tienen dirección. Si existe $A \rightarrow B$, no debe asumirse automáticamente que existe $B \rightarrow A$. Este error altera grados, vecindarios, recorridos y rutas.

Otro error frecuente es no controlar visitados. En presencia de ciclos, un recorrido sin visitados puede repetir vértices indefinidamente o procesar la misma zona del grafo muchas veces.

También es común confundir BFS con Dijkstra. BFS puede encontrar caminos con menor cantidad de saltos, pero no garantiza menor costo cuando las aristas tienen

pesos distintos. Dijkstra trabaja con costos acumulados y por eso es el algoritmo adecuado para grafos ponderados con pesos positivos.

Otro problema aparece al usar los vértices como si fueran índices consecutivos. En Java, los arreglos se indexan desde cero y de forma consecutiva, pero los vértices del grafo pueden tener etiquetas como 10, 25, 80 o 150. En esos casos se necesita un mapeo entre etiqueta real e índice interno.

Finalmente, debe evitarse escribir métodos extensos que hagan todo a la vez. La resolución de problemas con grafos requiere modularidad. Métodos como `vecinosSalientes`, `gradoPositivo`, `gradoNegativo`, `menorNoVisitado`, `indiceDe` o `copiarConjunto` permiten construir soluciones más claras, reutilizables y fáciles de analizar.

Síntesis conceptual

Resolver problemas con grafos exige pasar de la representación estructural a la estrategia algorítmica. El `GrafoTDA` ofrece operaciones primitivas para construir y consultar la estructura, pero las preguntas más relevantes suelen requerir métodos derivados. Calcular grados, obtener vecindarios, determinar alcanzabilidad, recorrer caminos o buscar rutas de menor costo implica combinar operaciones del TDA con estructuras auxiliares como conjuntos, colas, pilas, arreglos y diccionarios.

BFS y DFS permiten recorrer el grafo controlando ciclos mediante visitados. BFS explora por niveles y resulta adecuado para menor cantidad de saltos en grafos no ponderados. DFS explora en profundidad y resulta útil para analizar caminos completos y dependencias. Dijkstra agrega una dimensión de optimización: permite calcular caminos de menor costo cuando las aristas tienen pesos positivos.

La clave técnica está en comprender qué pregunta plantea el problema. Una conexión directa se resuelve con una consulta del TDA. Una conexión indirecta requiere recorrido. Una mejor ruta ponderada requiere comparar costos acumulados. Esta distinción ordena la elección del algoritmo, mejora la calidad de la implementación y permite justificar correctamente el costo de la solución.